

Kirigamizer: An Algorithm Proposal

Designing Kirigami (Cut + Fold) Patterns for Any Surface

Emre Dayangac

Proposal. A paper fold is an isometry, so it preserves intrinsic curvature: every interior point of a flat sheet keeps exactly 2π of material around it, and folding can only *redistribute* that angle, never change the total at a point. This single fact bounds what origami can do. At a convex vertex the target subtends *less* than 2π , leaving a surplus that a fold can hide, precisely Origamizer’s *tuck*. At a saddle vertex the target subtends *more* than 2π , which no interior point of a flat sheet can supply, so the fold is impossible and Origamizer can only *approximate* it (split the vertex, ε -extra). Negative curvature is therefore not an incidental weakness but the *single, structural* place where origami’s “isometry on a disk” assumption fails.

A *cut* lifts exactly that assumption. Where the sheet is cut, a point lies on the *boundary*, not the interior, so it is no longer pinned to 2π : its two lips may overlap (a cone, $< 2\pi$, for free) or splay open with a gusset (a saddle, $> 2\pi$). Cuts thus realize *both* signs of curvature, exactly and locally, supplying the one operation origami lacks, and only there. I therefore propose *Kirigamizer*: keep Origamizer’s pipeline unchanged, it is already strong everywhere except this obstruction, and introduce cuts precisely where curvature is negative or where tucking would be wasteful. The proposal contributes (i) a criterion that *determines where cuts must go*, derived from the intrinsic curvature of Q , and (ii) a construction that *reuses Origamizer’s modules* for everything cutting does not handle. Origamizer is the special case $\mathcal{C} = \emptyset$. Symbols: Q is the target (any genus g , any curvature, clean/tuck sides); $\varepsilon > 0$ the tolerance; output is a flat sheet P with cut set $\mathcal{C}$ and crease set $\mathcal{K}$, and a folding $f: P \rightarrow \mathbb{R}^3$.

1 Math

Angle defect (discrete Gaussian curvature) and the curvature budget (Gauss-Bonnet):

$$\delta(v) = 2\pi - \sum_i \alpha_i(v), \quad \sum_v \delta(v) = 2\pi \chi(Q) = 2\pi(2 - 2g). \quad (1)$$

Flat paper carries exactly 2π at every interior point, so the local action at a vertex is forced by the *sign* of $\delta(v)$:

$$\text{at } v : \quad \begin{cases} \delta(v) > 0 & \text{remove } \delta(v) \Rightarrow \textbf{dart (cut+overlap) or tuck,} \\ \delta(v) = 0 & \text{nothing,} \\ \delta(v) < 0 & \text{add } |\delta(v)| \Rightarrow \textbf{open slit or gusset.} \end{cases} \quad (2)$$

Correctness target (Origamizer’s containment, cuts allowed):

$$Q \subseteq f(P) \subseteq Q \cup \mathcal{O}_\varepsilon(Q) \implies d_H(Q, f(P)) \leq \varepsilon, \quad (3)$$

with $\mathcal{O}_\varepsilon(Q)$ the tuck-side ε -offset. A cut is *sealed* when its lips c_1, c_2 fold together, $d_F(f(c_1), f(c_2)) \leq \varepsilon$; else it is left *open* (a deliberate hole).

2 Why kirigami, not origami?

The case for cutting is not aesthetic; it is forced by geometry, efficiency, and topology.

(a) Origami can only *subtract* angle; kirigami can also *add* it. Rule (2) is asymmetric for a single uncut sheet. At a convex vertex ($\delta > 0$) the paper has surplus angle, which a fold can hide (tuck). At a saddle vertex ($\delta < 0$) the surface needs *more* than the 2π a flat interior point can supply, no fold can create it. So origami covers only *half* the curvature range; the deficit half is reachable only by a cut (open slit or gusset). This is the root reason, and it is exactly the operation Origamizer lacks.

(b) Negative curvature is the common case, not a corner case. Any target with a neck, a saddle, a handle, or genus $g > 0$ has $\delta < 0$ vertices, and by (1) a genus- g surface carries total curvature $2\pi(2 - 2g)$, so for $g \geq 2$ the surface is *net negatively curved*. For all of these, an origami-only Origamizer must fall back on its ε -extra construction, which *never folds the surface exactly*: it splits each saddle vertex and approximates it within ε . Kirigami realizes those vertices *exactly* with a cut.

(c) Cutting is dramatically more material-efficient. On the Stanford bunny (374 triangles) Origamizer puts only $\approx 22.3\%$ of the paper onto the visible surface, nearly 78% is hidden tuck, a $\sim 2:1$ scale penalty per dimension. The tuck behind a convex vertex grows with its surplus $\delta(v)$ and the size of the surrounding facets. A *dart* discards the same surplus through a thin slit whose lips merely overlap by angle $\delta(v)$, no deep buried pleat. The saved area scales with the total positive curvature $\sum_v \max(\delta(v), 0)$, which is large for any spiky or detailed model.

(d) Fewer layers $\Rightarrow$ actually foldable. Tucks stack many paper layers at each vertex; the bunny’s pattern took a ~ 10 -hour hand fold and is, in the authors’ words, likely impossible to design by hand. Cuts collapse those layer stacks, so the result is thinner and foldable from stiff stock (cardstock, sheet metal, composites) where a thick origami tuck simply cannot close.

(e) Topology and function come free with cuts. A single uncut sheet is a disk: holes, handles, and non-orientable targets must first be *cut* into a disk before Origamizer even starts. Kirigami takes those cuts as part of the design. Moreover, whole classes of useful structures, auxetic / negative-Poisson metamaterials, stretchable electronics, deployable kirigami honeycombs, are *defined* by their cuts and have no origami-only realization at all.

Summary. Origami is the special case $\mathcal{C} = \emptyset$, optimal only when the target is (nearly) developable and convex. The moment the target has saddles, handles, fine detail, or a functional cut pattern, i.e. most real targets, kirigami is not a stylistic choice but the *exact, efficient, manufacturable* one. The rest of this proposal makes that switch precise: §4 decides where to cut, and §5 keeps everything origami still does best.

3 Input: conditioning and file format

The whole method rests on the angle defect $\delta(v) = 2\pi - \sum_i \alpha_i(v)$, which is only well defined on a *manifold, flat-faceted, coarse* mesh with shared vertices. So the raw 3D model must first be conditioned, and the choice of file format matters.

STL vs. STEP. *Prefer STEP (or a topological mesh: OBJ / PLY / OFF); avoid raw STL.*

- **Connectivity.** $\delta(v)$ needs vertices *shared* between faces. STL is triangle soup: duplicated vertices, no adjacency, so δ is undefined until the mesh is welded and its topology rebuilt. STEP is a B-rep: exact faces, edges, and adjacency are already present.
- **Feature edges.** Cut routing (6) wants to follow real edges (to stay seamless). STEP carries them; STL does not.
- **Units / scale.** The tolerance ε and any fabrication need true units. STEP stores units; STL is unitless and often inconsistent.
- **Curvature placement.** STEP surfaces are smooth (NURBS), so curvature is continuous; you must *facet* them, which is exactly the step that decides where curvature concentrates (the cone vertices). On a dense STL the opposite problem bites: almost every vertex has $\delta \neq 0$, so (4) would cut *everywhere* unless the mesh is first decimated to a coarse flat-faceted target.

Decision: STEP is the input format. It carries exact topology, feature edges, and units, and lets us choose the faceting (hence the cone vertices) ourselves, which is precisely what the cut planner needs. The fixed pipeline is

STEP $\rightarrow$ condition to coarse manifold mesh $Q \rightarrow$ half-edge internally $\rightarrow$ **FKLD** + **SVG/DXF**,

where FKLD (a FOLD superset: folds, cuts, molecules, and material together, §9) runs directly in the bar-and-hinge simulator and SVG/DXF (cut vs. crease on separate layers) drives a cutter/plotter. A mesh (OBJ/PLY/OFF) is accepted as a fallback, and raw STL only after weld + manifold repair + decimation.

Algorithm 1: IMPORTANDCONDITION(M): STEP model $\rightarrow$ usable target Q (Stage 0)

Input: STEP B-rep model M (fallback: a mesh OBJ/PLY/OFF/STL).

Output: Coarse, manifold, flat-faceted target Q with consistent units and clean/tuck sides.

```

1 if  $M$  is STEP (B-rep) then
2   | read units and feature edges; tessellate each face into planar facets at the desired
   | coarseness
3 else  $\triangleright$  fallback:  $M$  is a mesh
4   | weld coincident vertices; rebuild edge/face adjacency
5 Repair to a 2-manifold (fill/zip cracks, fix orientation, remove degenerates)
6 Decimate / remesh to a coarse flat-faceted model so curvature concentrates at a chosen
   vertex set (cone vertices)
    $\triangleright$  otherwise  $\delta \neq 0$  almost everywhere  $\Rightarrow$  (4) cuts everywhere
7 Record units; assign the clean side (visible) and tuck side (hidden)
8 return  $Q$ 

```

4 Where the cuts go (the determination)

Cut placement is *derived*, not guessed, in three steps: *necessity*, *connection*, *routing*.

(1) Necessity: which vertices force a cut. A surface flattens isometrically to the plane iff it is a simply connected *developable disk*: no interior vertex with nonzero defect, and genus 0. Cutting moves curvature from the interior to a boundary, where it is realized as an overlap (dart) or a gap (slit). Hence the hard constraint

$$v \in \mathcal{C} \iff \delta(v) \neq 0, \quad (4)$$

i.e. *every defect vertex must become a boundary vertex of some patch*. By (1) the *total* signed curvature is fixed by topology, so the amount of cutting is topologically forced, the proposal only chooses *how* to realize it. In addition, for genus $g > 0$ a set of $2g$ non-contractible loops H must be cut to reduce the genus to 0.

(2) Connection: making patches developable. Isolated punctures do not flatten; the cut set must be a *graph* $\mathcal{C}$ such that every component of $Q \setminus \mathcal{C}$ is a disk whose interior is flat. Equivalently, $\mathcal{C}$ must connect every defect vertex (4) to a patch boundary, and include the handle loops H . This is a Steiner-forest / cut-tree condition on the mesh:

$$\text{required terminals} = \{v : \delta(v) \neq 0\} \cup \partial Q, \quad \mathcal{C} \supseteq H. \quad (5)$$

(3) Routing: the exact location, by optimization. Among all $\mathcal{C}$ satisfying (1)-(2), pick one minimizing a cost that trades cut length against *visibility* (cuts on the clean side are penalized; cuts on edges of Q or on the tuck side are cheap):

$$\min_{\mathcal{C}} \sum_{e \in \mathcal{C}} \left(\text{len}(e) + \lambda \text{vis}(e) \right) \quad \text{s.t. (4), patches developable, } \mathcal{C} \supseteq H, \quad (6)$$

with λ the seam-visibility weight. Practically: route $\mathcal{C}$ along *mesh edges of* Q (so the clean side stays seamless), connect defect vertices by *shortest paths / geodesics* (minimizing length), and push unavoidable interior cuts onto the *tuck side*. Finally, the action at each cut vertex follows the sign rule (2); for $\delta(v) > 0$ there is a free choice *dart vs. tuck* (§5), resolved inside the optimization (6) by comparing a dart's seam cost against an Origamizer tuck.

Algorithm 2: PLAN CUTS(Q, λ): decide where the cuts go (§4)

Input: Surface Q ; seam-visibility weight λ .

Output: Cut set $\mathcal{C}$ (each cut tagged dart/slit/gusset) and tuck set Θ of vertices to be handled by Origamizer.

```

1 for each vertex  $v$  of  $Q$  do
2    $\delta(v) \leftarrow 2\pi - \sum_i \alpha_i(v)$ 
3    $D \leftarrow \{v : \delta(v) \neq 0\}$ 
                                      $\triangleright$  necessity (4): only these vertices force a cut
4    $H \leftarrow 2g$  non-contractible loops of  $Q$ 
                                      $\triangleright$  cut handles so each patch has genus 0
5    $\mathcal{C} \leftarrow$  min-cost graph on mesh edges linking  $D \cup \partial Q$ , plus  $H$ , minimizing (6)
                                      $\triangleright$  prefer edges of  $Q$ ; shortest paths; hide on the tuck side
6    $\Theta \leftarrow \emptyset$ 
7   for each  $v \in D$  do
8     if  $\delta(v) < 0$  then
9       tag  $v$ 's cut slit (open) or gusset of angle  $|\delta(v)|$ 
10    else if  $\delta(v) > 0$  and darting is cheaper than tucking then
11      tag  $v$ 's cut dart of angle  $\delta(v)$ 
12    else  $\triangleright \delta(v) > 0$ , seamlessness preferred
13      move  $v$  from  $\mathcal{C}$  to  $\Theta$ 
14 return  $(\mathcal{C}, \Theta)$ 

```

5 What it reuses from Origamizer

Kirigamizer is not a rewrite: every step that cutting does not handle is an Origamizer module, invoked unchanged. Cutting is a *front-end* that removes exactly the curvature Origamizer struggles with; Origamizer then does the rest.

Origamizer part	How Kirigamizer uses it
Angle-defect / curvature analysis (§3)	drives cut placement (4)-(2) instead of the tuck-proxy inset
Vertex-unfolding (§4)	flattens each developable patch in SEAMEDUNFOLD
Tutte embedding (§5)	packs the unfolded patches into one sheet P
Streams: rivers & brooks (§5)	become <i>seams/tabs</i> that rejoin lips, the same channels, run in reverse to <i>deliver</i> material to $\delta < 0$ vertices
Tuck proxy / tucking molecules (§3, §7)	invoked <i>locally</i> for the $\delta > 0$ vertices in Θ we choose to tuck (keeps the clean side seamless there)
Site placement + Voronoi folding (§6-§7)	generates the actual crease pattern for each tucked region
Guarantees: watertight / seamless / ε -extra	generalized to <i>seam</i> -watertight / <i>seam</i> -seamless / ε -extra (3)

The division of labor: **cuts** take the cases Origamizer cannot do exactly ($\delta < 0$, genus, nonorientable); **Origamizer's tucking** takes the cases it already does well ($\delta > 0$ where a seamless

single sheet is wanted). The *dart-vs-tuck* switch in PLANcuts is exactly the dial between the two regimes.

6 Top-level driver

Algorithm 3: KIRIGAMIZER(M, ε, λ): design a cut+fold pattern for model M

Input: STEP model M (B-rep); tolerance $\varepsilon > 0$; seam-visibility weight λ .

Output: Sheet P with cuts $\mathcal{C}$, creases $\mathcal{K}$, folding f with $f(P) \subseteq Q \cup \mathcal{O}_\varepsilon(Q)$.

```

1  $Q \leftarrow \text{ImportAndCondition}(M)$ 
    $\triangleright$  Stage 0 (§3): weld/repair/coarsen to a flat-faceted manifold
2  $(\mathcal{C}, \Theta) \leftarrow \text{PlanCuts}(Q, \lambda)$ 
    $\triangleright$  §4: where the cuts go; what stays tucked
3  $(\{U_k\}, \mathcal{K}) \leftarrow \text{SeamedUnfold}(Q, \mathcal{C})$ 
    $\triangleright$  cut into developable patches, flatten isometrically
4  $(P, \mathcal{C}, \mathcal{K}) \leftarrow \text{RouteSeams}(\{U_k\}, \mathcal{C}, \mathcal{K})$ 
    $\triangleright$  pack to one sheet; route seams/tabs
5  $\text{EmitPattern}(P, \mathcal{K}, \mathcal{C}, \Theta)$ 
    $\triangleright$  write crease+cut pattern; tuck  $\Theta$  via Origamizer
6  $f \leftarrow \text{SimulateVerifyOptimize}(P, \mathcal{K}, \mathcal{C}, Q, \varepsilon)$ 
    $\triangleright$  GPU-fold, check  $d_H \leq \varepsilon$ , refine
7 return  $(P, \mathcal{C}, \mathcal{K}, f)$ 
```

7 Stage algorithms

Stage 1: Plan cuts. Given in PLANcuts (§4); replaces Origamizer’s tuck proxy.

Stage 2: Seamed unfolding (reuses vertex-unfolding). With the cuts in place, Q separates into developable patches that flatten *isometrically*, no ε -approximation is needed for saddle vertices, since their excess has been absorbed by a cut.

Algorithm 4: SEAMEDUNFOLD($Q, \mathcal{C}$): cut into patches and flatten

Input: Surface Q ; cut set $\mathcal{C}$.

Output: Planar patches $\{U_k\}$; crease set $\mathcal{K}$.

```

1  $\{Q_k\} \leftarrow$  connected components of  $Q$  after cutting along  $\mathcal{C}$ 
2  $\mathcal{K} \leftarrow \emptyset$ 
3 for each patch  $Q_k$  do
4    $U_k \leftarrow$  vertex-unfold  $Q_k$  isometrically into the plane
    $\triangleright$  developable  $\Rightarrow$  isometric immersion exists
5   while  $U_k$  self-overlaps do
6     Add a relief cut across  $Q_k$  and re-split; re-unfold
      $\triangleright$  an immersion need not embed; relief cuts restore an embedding
7   Add the interior fold edges of  $U_k$  (with M/V assignment) to  $\mathcal{K}$ 
8 return  $(\{U_k\}, \mathcal{K})$ 
```

Stage 3: Seam & tab routing (reuses streams). Pack the patches into one sheet and connect the lips that must rejoin: a zero-width *seam* (brook) for sealed cuts, or a thickened *tab/gusset strip* (river) that carries extra material to $\delta < 0$ vertices; open cuts stay as boundaries of P .

Algorithm 5: ROUTESEAMS($\{U_k\}, \mathcal{C}, \mathcal{K}$): pack and connect

Input: Patches $\{U_k\}$; cut set $\mathcal{C}$; crease set $\mathcal{K}$.

Output: Sheet P ; updated $\mathcal{C}, \mathcal{K}$.

```

1  $P \leftarrow$  pack  $\{U_k\}$  into one sheet (Tutte-embed patch-adjacency graph)
2 for each cut  $c \in \mathcal{C}$  marked sealed do
3   if  $c$  adds angle (a  $\delta < 0$  vertex) then
4     | route a tab/gusset strip (river) between the lips; add folds to  $\mathcal{K}$ 
5   else
6     | route a zero-width seam (brook) matching the lips
7 Leave every open cut as a free boundary of  $P$ 
8 return  $(P, \mathcal{C}, \mathcal{K})$ 

```

Stage 4: Emit pattern (reuses site placement & Voronoi folding). Write mountain/valley creases plus cut edges in the **FKLD** format (§9); for each tucked vertex, generate its Origamizer tuck locally and merge.

Algorithm 6: EMITPATTERN($P, \mathcal{K}, \mathcal{C}, \Theta$): write the crease+cut pattern

Input: Sheet P ; crease set $\mathcal{K}$; cut set $\mathcal{C}$; tuck set Θ .

Output: An FKLD file over P (§9).

```

1 for each edge  $k \in \mathcal{K}$  do
2   | mark  $k$  mountain/valley/facet with target fold angle
3 for each edge  $c \in \mathcal{C}$  do
4   | mark  $c$  as a cut (boundary) edge
5 for each vertex  $v \in \Theta$  do
6   |  $\mathcal{K} \leftarrow \mathcal{K} \cup \text{OrigamizerTuck}(v)$ 
   |  $\triangleright$  Origamizer §6-§7: local Voronoi tuck (geodesic-vertex sites)
7 Export  $(P, \mathcal{K}, \mathcal{C})$ 

```

Stage 5: Simulate, verify, optimize (replaces the constructive proof). The GPU bar-and-hinge solver treats cuts as first-class edges (§8: full cuts split the mesh, partial cuts soften the hinge by $(1 - \text{cutRatio})$), and a driven-boundary forward process steers it to the goal mesh M_0 ; iterate until the folded state lies within ε of Q .

Algorithm 7: SIMULATEVERIFYOPTIMIZE($P, \mathcal{K}, \mathcal{C}, Q, \varepsilon$)

Input: Pattern $(P, \mathcal{K}, \mathcal{C})$; target Q ; tolerance ε .

Output: Folding $f: P \rightarrow \mathbb{R}^3$ with $d_H(Q, f(P)) \leq \varepsilon$.

```

1 repeat
2   |  $f \leftarrow \text{FOLDSOLVER.SOLVE}(P, M_0, 1, \text{iters})$ 
   |  $\triangleright$  AKDE explicit solver; driven boundary to  $M_0$  (§8)
3   |  $r \leftarrow d_H(Q, f(P))$ 
4   | if  $r > \varepsilon$  or strain/self-intersection too high then
5     | Adjust: cut routing  $\lambda$ , dart-vs-tuck in  $\Theta$ , fold angles, scale
6 until  $r \leq \varepsilon$ 
7 return  $f$ 

```

Correctness target. **Seam-seamless:** each facet of Q is one uncut face on the clean side (cuts on edges / tuck side). **Seam-watertight:** every cut is lip-matched ($d_F \leq \varepsilon$) or tuck-hidden, so the

clean side is gap-free. ε -**extra**: $f(P) \subseteq Q \cup \mathcal{O}_\varepsilon(Q)$, hence $d_H \leq \varepsilon$ (3).

8 Simulation: AKDE bar-and-hinge with driven boundary

Stage 5 uses the simulator from **AKDE** (`kirigami/sim`): the explicit *bar-and-hinge* solver of Ghassaei, Demaine, and Gershenfeld, extended for kirigami. Its CPU force math (`forces.ts`) is the unit-testable twin of the GPU GLSL passes; the mesh is stored struct-of-arrays (one texel per node/edge/face). Masses are unit ($m \equiv 1$). AKDE defaults: $EA = 20$, $k_{\text{fold}} = k_{\text{facet}} = 0.7$, $k_{\text{face}} = 0.2$, damping $\zeta \approx 0.45$ (AKDE runs overdamped, $\zeta \approx 1$, to settle the flat→cone buckling), design angles $\theta_M \approx 1.2$, $\theta_V \approx 2.9$ (valley near π so each molecule tucks nearly flat, collapsing the flat-net perimeter radius s onto the base radius R).

Signed dihedral fold angle (branch-safe). With $\hat{e}$ the unit crease-edge vector and n_1, n_2 the adjacent face normals,

$$\theta = \text{atan2}((n_1 \times \hat{e}) \cdot n_2, n_1 \cdot n_2), \quad \gamma = \pi - \theta, \quad (7)$$

unwrapped against the previous step so it tracks continuously through $\pm\pi$ (γ is the geometric dihedral). This signed `atan2` form (DETC2019 Eq. 5) stays valid through the full fold range, unlike a $\sin \gamma$ closed form.

Axial bar + neighbour damping (per edge). With $\hat{d}$ the unit edge vector, length l , rest l_0 , $k_a = EA/l_0$:

$$F_{\text{ax}} = k_a (l - l_0) \hat{d}, \quad F_{\text{damp}} = c (v_b - v_a), \quad c = 2\zeta \sqrt{k_a m_{\text{min}}}, \quad (8)$$

i.e. damping is *relative* between the two endpoints, not a global drag.

Crease torque on four nodes. For the two wing vertices $w \in \{1, 2\}$ (opposite the crease, with face normal $\hat{n}_w$ and moment arm h_w to the crease line) and the two crease nodes split by along-crease fractions coef_w , scaled by the fold slider $\text{fp} \in [0, 1]$:

$$\Phi = k_c (\text{fp} \cdot \theta_{\text{tgt}} - \theta), \quad F_{\text{wing } w} = \frac{\Phi}{h_w} \hat{n}_w, \quad (9)$$

with momentum-conserving reactions on the crease nodes weighted by $(1 - \text{coef}_w)$ and coef_w .

Kirigami coupling (DETC2019-97557, Eq. 6: $Kd = Fx$). A cut softens the hinge in proportion to lost ligament:

$$k_c = (1 - \text{cutRatio}) l_0 k, \quad k \in \{k_{\text{fold}}, k_{\text{facet}}\}, \quad (10)$$

so $\text{cutRatio} = 0$ is exactly Gershenfeld and $\text{cutRatio} = 1$ is a free flap. Fully cut (major/minor) edges are simply *absent as creases*: their lips are independent nodes carrying only a boundary bar, which is what lets holes and darts open. (This is AKDE’s actual mechanism, replacing the earlier “seam spring” sketch.)

Face interior-angle spring. For each triangle angle α with flat value α_0 , a torque from $k_{\text{face}}(\alpha_0 - \alpha)$ applied via the gradient $\text{cross}(n, e)/|e|^2$ on the three nodes, resisting in-plane shear.

Explicit integration. Forward Euler at unit mass, with the stable step bounded by the stiffest (axial) mode:

$$v \leftarrow v + \frac{F}{m} \Delta t, \quad p \leftarrow p + v \Delta t, \quad \Delta t = \frac{0.9}{2\pi \omega_{\max}}, \quad \omega_{\max} = \max \sqrt{k_a/m_{\min}}. \quad (11)$$

State lives in GPU textures; one thread per element, so per-step cost is roughly constant until node count exceeds core count, then linear.

Driven-boundary forward process (the design coupling, DETC §3.2). This is how the simulator is *steered* to the designed shape, not merely checked against it. Boundary nodes are driven kinematically toward the goal mesh M_0 (the folded target Q) and pinned, so the force passes leave them in place while the interior relaxes:

$$p_i = \text{rest}_i + (\text{goal}_i - \text{rest}_i) \cdot \text{fp}, \quad i \in \text{driven}. \quad (12)$$

The slider fp is eased *quasi-statically*, $\text{fp} \leftarrow \text{fp} + (\text{target} - \text{fp}) \cdot 0.004$ per step, because a fast ramp injects energy the explicit integrator cannot dissipate.

Algorithm 8: FOLDSOLVER.SOLVE($P, M_0, \text{target}, \text{iters}$): AKDE explicit kirigami solver (§8)

Input: Bar-hinge model of P (beams, creases with θ_{tgt} and cutRatio , faces); goal mesh M_0 ; target fold fraction; step count.

Output: Folded positions f (boundary driven to M_0 , interior relaxed).

```

1 Build model: split mesh at major/minor cuts; set  $k_c$  by (10); mark driven boundary nodes
2  $\Delta t \leftarrow 0.9/(2\pi \omega_{\max})$ ;  $\text{fp} \leftarrow 0$ ;  $v \leftarrow 0$ 
3 for  $\text{iters}$  steps do
4    $\text{fp} \leftarrow \text{fp} + (\text{target} - \text{fp}) \cdot 0.004$  ▷ quasi-static ease
5   drive boundary by (12) for  $i \in \text{driven}$ 
6   pass 1: face normals  $n_t$ 
7   pass 2: fold angles  $\theta_e$  (7) (unwrapped)
8   pass 3:  $F \leftarrow F_{\text{ax}} + F_{\text{damp}} + F_{\text{crease}} + F_{\text{face}}$ 
9   pass 4: integrate (11); skip fixed/driven nodes
10 return  $f \leftarrow p$ 
```

9 FKLD output format

EMITPATTERN writes **FKLD** (Flexible Kirigami list Datastructure), AKDE’s format: a strict *superset* of FOLD 1.2 (Demaine, Ku, Lang). Every FKLD file is a valid FOLD file, so any FOLD tool renders the crease pattern and ignores the kirigami extensions, which live under the reserved `fkld:` namespace. The standard FOLD arrays carry the geometry: `vertices_coords`, `edges_vertices`, `edges_assignment` (M/V/B/U/F plus C = cut), `edges_foldAngle`, `faces_vertices`, `faceOrders` (tuck stacking), and `file_frames` (flat / target / folded states).

The kirigami extensions (FKLD v0.1 registry):

- `fkld:edges_cutType`: subtype of each C edge, one of `major`, `minor`, `seam`, `dart`, `auxetic`, `vent`, `tab`.
- `fkld:edges_moleculeTheta` / `_moleculeWidth` / `_moleculeDepth`: tucking-molecule angle $\theta(i, j)$, chord width $w(i, j)$, and tuck depth behind the face (Tachi 2010).

- `fkld:edges_dihedralTarget`: folded dihedral target (rad), i.e. the θ_{tgt} consumed by the solver (7).
- `fkld:vertices_curvatureClass` (positive/negative/flat/boundary), `fkld:vertices_angleDefect` $G_v = 2\pi - \sum \theta$ (1), and `fkld:vertices_reliefStrategy` (molecule/cut/hybrid/none).
- `fkld:faces_materialId` / `_thickness` / `_structuralRole` / `_panelId` for fabrication.

The correspondences close the loop: `vertices_reliefStrategy` is the dart-vs-tuck dial of PLAN-CUTS (§4); `vertices_angleDefect` is (1); the molecule keys carry the Origamizer tuck (§5); `edges_dihedralTarget` feeds the solver. For plain-FOLD tools, `filter.splitCuts` downgrades C to B (lossy but legal; the `fkld:` annotations are preserved). A schematic FKLD for one darted positive-curvature vertex:

```
{ "file_spec":1.2, "file_creator":"AKDE/Kirigamizer", "unit":"mm",
  "file_classes":["creasePattern"],
  "vertices_coords":[[0,0],[10,0],[10,10],[0,10],[5,5]],
  "edges_vertices":[[0,1],[1,2],[2,3],[3,0],[4,1],[4,3]],
  "edges_assignment":["B","B","B","B","V","C"],
  "edges_foldAngle":[0,0,0,0,150,0],
  "faces_vertices":[[0,1,4],[1,2,3,4],[3,0,4]],
  "fkld:edges_cutType":[null,null,null,null,null,"dart"],
  "fkld:vertices_curvatureClass":
    ["boundary","boundary","boundary","boundary","positive"],
  "fkld:vertices_angleDefect":[0,0,0,0,1.05],
  "fkld:vertices_reliefStrategy":
    ["none","none","none","none","hybrid"]} }
```

10 Preconditions and limitations (honest scope)

What the pipeline does and does not guarantee:

- **Coarse flat-faceted input required.** The defect rule (4) assumes curvature lives at vertices. The STEP model is faceted and conditioned to a coarse mesh in Stage 0 (§3); the faceting coarseness sets where curvature concentrates, and the output quality inherits that choice.
- **Open kirigami is fully general; sealed kirigami is not (yet).** Negative-curvature vertices are always realizable as *open* slits (deliberate holes). The *sealed* route (a gusset that delivers material while staying watertight) has no proof of a valid, non-crossing layer ordering in general; it is heuristic, checked in Stage 5.
- **Unfolding may need extra cuts.** A developable patch immerses isometrically but need not embed; the relief loop in SEAMEDUNFOLD adds cuts until it embeds, so the final cut count can exceed the curvature minimum.
- **No constructive flat-foldability guarantee.** Mountain/valley assignment and a collision-free layer order are produced heuristically and validated by the simulator, not proven (unlike Origamizer §7).
- **Stage 5 is a search, not a solver.** The verify/refine loop has no convergence guarantee; it terminates when $d_H \leq \varepsilon$ or on a budget.

- **Zero-thickness model.** Material thickness, laser kerf, fold allowance/score lines, and joining tabs for sealed seams are out of scope here and must be added for real fabrication.

Bottom line. As written, the method reliably turns a conditioned, coarse 3D model into an *open*-kirigami pattern; the watertight *sealed* variant and guaranteed flat-foldability are the open parts, deferred to the simulator and to future work.